

PROMETHEUS & GRAFANA

Hands-On Learning Notes

A beginner-friendly walkthrough of setting up open-source infrastructure monitoring and alerting on AWS EC2 — documented step-by-step while practicing in the AWS Console.

Prepared by Lasya Mandapati

Topics Covered

- Node Exporter — exposing system metrics from a Linux server
- Prometheus — installing it and scraping metrics on a schedule
- Grafana — connecting to Prometheus and building live dashboards
- Reading raw metrics — understanding what /metrics actually shows
- Alertmanager — defining alert rules and routing them to Slack
- AWS Security Groups — opening only the ports each tool needs

1 Prometheus & Grafana — Overview

What are Prometheus and Grafana? Prometheus is an open-source tool that collects and stores numbers about a system over time — things like CPU usage, memory, and disk space. Grafana is a dashboard tool that connects to Prometheus and turns those stored numbers into graphs, gauges, and tables that are far easier to read than raw data. Used together, they let you see the health of a server at a glance, and get alerted the moment something goes wrong.

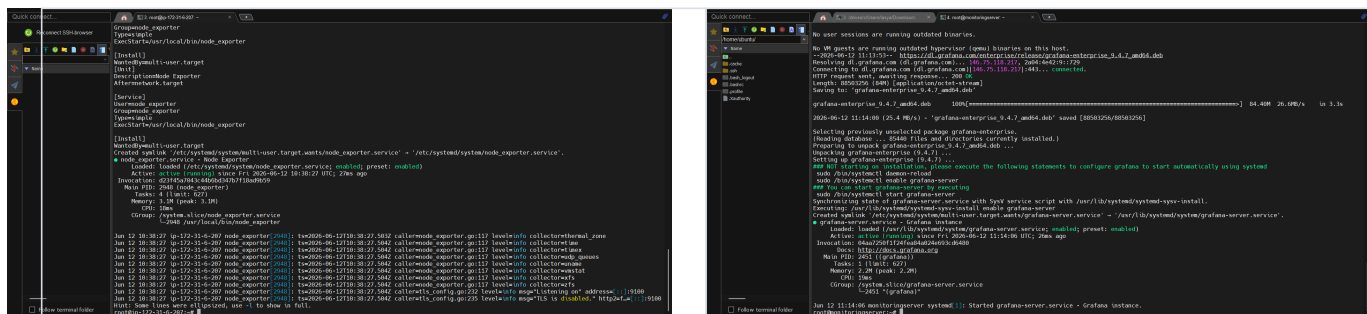
Think of it as three connected pieces: **Node Exporter** runs on the server being watched and publishes its stats at a web address ending in `:9100/metrics`. **Prometheus** visits that address every few seconds and pulls in the latest numbers, storing them with a timestamp. **Grafana** then queries Prometheus and draws the results as live charts. A fourth piece, **Alertmanager**, watches the incoming data for warning conditions and pushes an instant notification — in this project, to Slack — the moment one is met.

The rest of these notes walk through each stage in the order it was actually set up: installing the tools, reading the metrics they produce, building dashboards, wiring up alerts, and finally locking down the server with the correct AWS Security Group rules.

2 Installation — Node Exporter & Grafana

Both tools are installed directly on the EC2 server through the terminal, using MobaXterm as the SSH client. Each is set up as a **systemd service**, so the server itself starts it on boot and restarts it automatically if it ever crashes.

Node Exporter is installed first and confirmed **active (running)**, listening on port **9100** — the standard port it uses to publish metrics. Grafana is then downloaded directly from its official server as a `.deb` package, installed with the system package manager, enabled to start on boot, and started — the terminal output confirms it too is **active (running)**, ready to be opened in a browser.



The image shows two terminal windows side-by-side. The left window displays the installation of Node Exporter using dpkg, followed by the creation and enabling of a systemd service named 'node_exporter.service'. The right window shows the download of Grafana Enterprise using wget, followed by the creation and enabling of a systemd service named 'grafana-server.service'.

```
Left: Node Exporter's systemd service created, enabled, and running on the monitored EC2 instance. Right: Grafana Enterprise downloaded, unpacked, and started as a systemd service on the monitoring server.
```

3 Instance Metrics — Reading the Raw Data

Once Node Exporter is running, it exposes a live, plain-text page at `http://<server-ip>:9100/metrics`. This is the raw feed Prometheus reads from — not meant to be pretty, but the foundation everything else is built on. Each line is one measurement: a line starting with `#` HELP explains what the metric means, and the line right after it shows the current value.

A few examples from the page below: **go_goroutines** is the number of background tasks currently running inside Node Exporter itself; **go_memstats_heap_inuse_bytes** is how much memory is actively in use at that moment; **go_gc_duration_seconds** is how long the program's automatic memory clean-up pauses last.

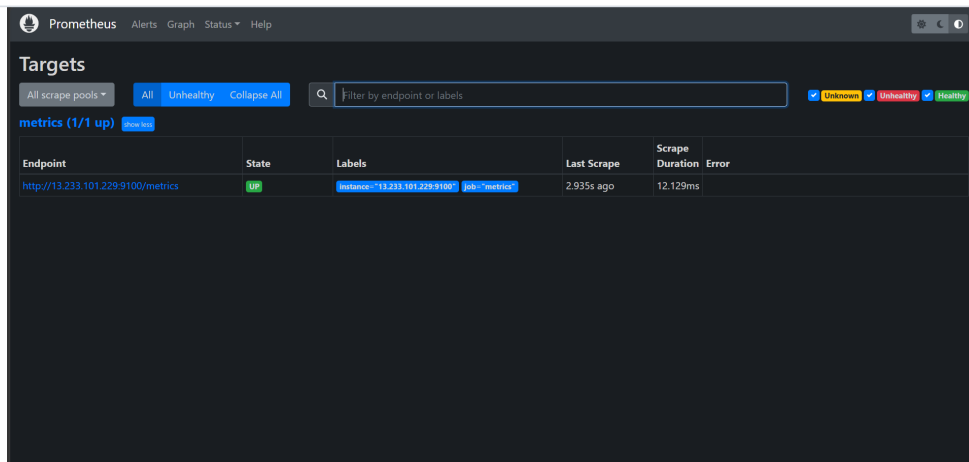
```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.19.3"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 1.092448e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 1.092448e+06
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.445427e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 717
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 8.29614e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 1.092448e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 1.548288e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 2.351184e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 8025
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 1.548288e+06
# HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system.
# TYPE go_memstats_heap_sys_bytes gauge
```

Raw output from the Node Exporter metrics endpoint — the same data Prometheus scrapes every few seconds and stores for later use.

Prometheus never shows this raw page to end users. It repeatedly downloads it in the background, timestamps every value, and stores it in its own database — turning a single snapshot into a searchable history that dashboards and alerts can be built on top of.

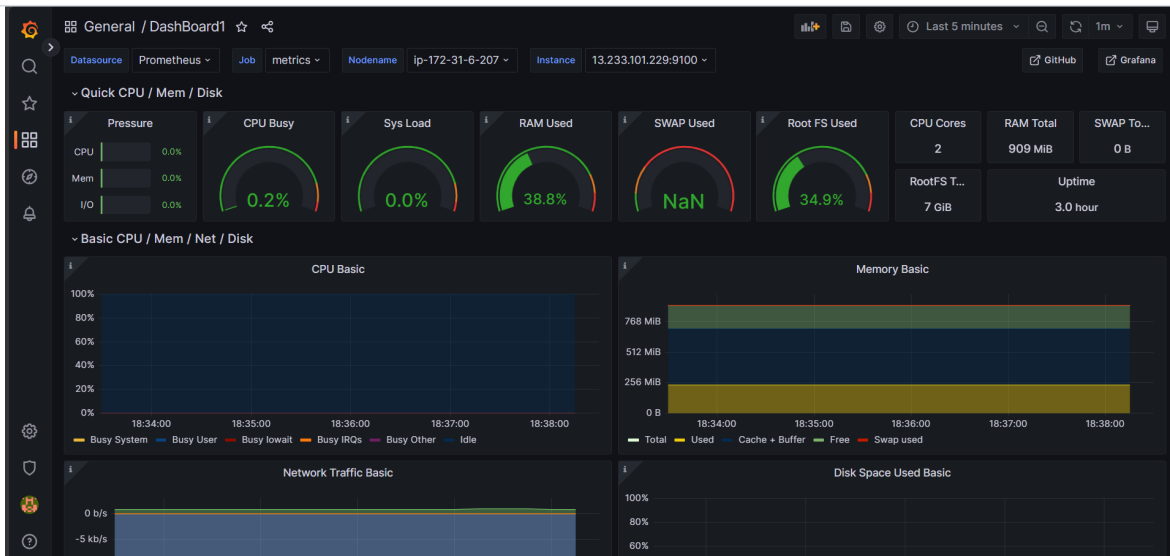
4 Dashboards — Visualizing Server Health

Confirming Prometheus is collecting data. Before building a dashboard, the Prometheus **Targets** page confirms the server can actually be reached. A green **UP** badge means the connection is healthy and metrics are being collected successfully; a red status would point to a misconfiguration such as a closed firewall port or a stopped Node Exporter service.



Prometheus "Targets" page showing the monitored instance is UP and being scraped successfully every few seconds, with zero errors.

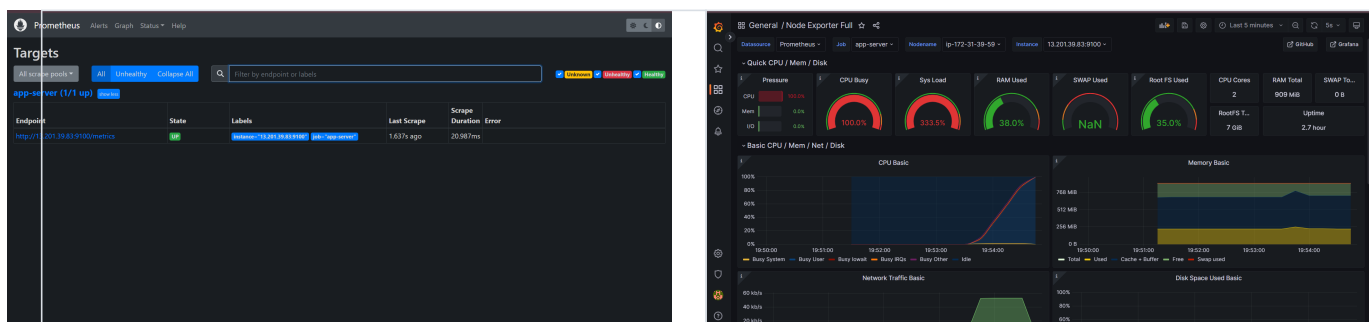
Turning numbers into visuals with Grafana. With Prometheus collecting data, Grafana is connected to it as a data source and used to build a live dashboard — quick-glance **gauges** at the top (CPU, RAM, swap, disk) for an instant health check, and **time-series graphs** below for spotting trends over time, including network traffic in and out of the server.



Grafana dashboard displaying real-time CPU, memory, network, and disk statistics pulled directly from Prometheus.

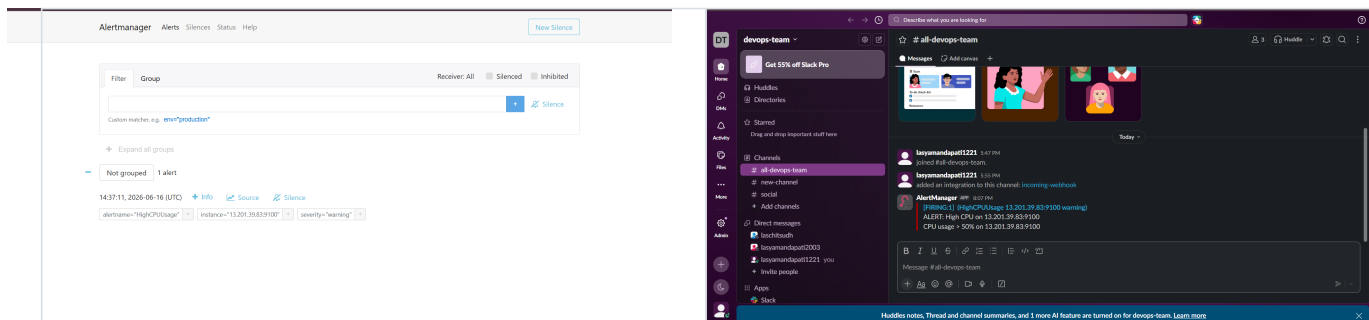
5 Alert Manager — Getting Notified Instantly

Watching a dashboard all day isn't practical, so a second server (job name **app-server**) is added to test end-to-end alerting: Prometheus picks it up as a new target and confirms it is UP, exactly as with the first server. Its CPU is then deliberately pushed to 100% — the Grafana dashboard immediately reflects the spike, giving visual confirmation the pipeline is working before the alert even fires.



Left: Prometheus Targets page confirming the second server (app-server) is being monitored successfully. Right: the live dashboard capturing a simulated CPU spike to 100%, used to test the alert rule.

A rule defined in Prometheus (HighCPUUsage) continuously checks whether CPU usage stays above 50%. Once triggered, Alertmanager groups the alert and routes it to its configured destination — a Slack channel, via an **incoming webhook** (a private URL that lets Alertmanager post messages directly into a chosen channel). The message includes the alert name, the affected server, and the exact condition that was breached.



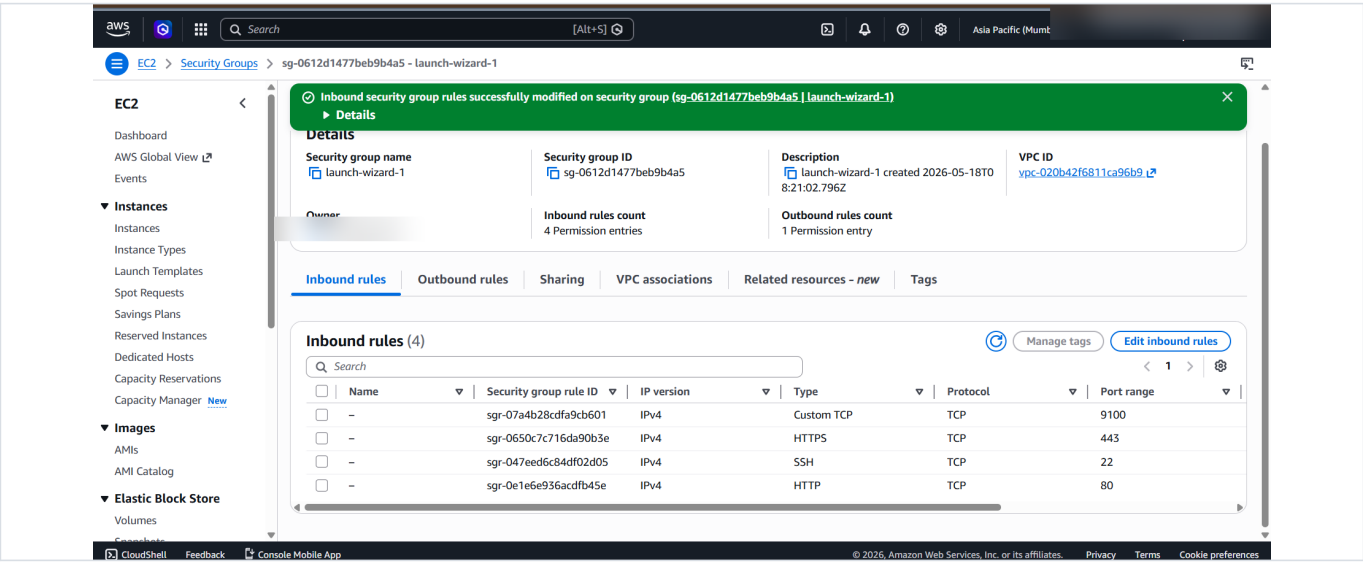
Left: Alertmanager's web interface showing the HighCPUUsage alert actively firing, tagged with the affected instance and a 'warning' severity level. Right: the Slack channel receiving the automatic notification the instant high CPU usage was detected.

6 Security Groups — Locking Down the Server

Running monitoring tools is only half the job — the server also needs to be protected so only the right traffic can reach it. On AWS this is controlled by a **Security Group**, a virtual firewall attached to the EC2 instance. By default everything is blocked; each inbound rule is one exception that allows a specific type of traffic in, on a specific port.

Port	Protocol	Purpose
22	SSH (TCP)	Secure terminal access to manage the server remotely.
80	HTTP (TCP)	Standard, unencrypted web traffic.
443	HTTPS (TCP)	Secure, encrypted web traffic — used for Grafana's web interface.
9100	Custom TCP	Lets Prometheus reach the Node Exporter metrics endpoint.

The AWS Console confirmation below shows these four rules successfully applied to the security group attached to the monitoring instance. Account-identifying details have been blurred out for privacy. In a real production environment, port 22 (SSH) would typically be restricted further — allowed only from a specific trusted IP address rather than left open to everyone.



AWS EC2 Security Group console confirming the four inbound rules (SSH, HTTP, HTTPS, and the Node Exporter port 9100) are active on the monitoring server.

End of notes. All screenshots in this document were cropped and redacted for privacy before publishing.